# Coding Question Examples for CV1

(Last updated: 29 Sep 2026)

Below are illustrative examples of how coding may be tested on the exam. Please note that these are not an exhaustive list of question styles or difficulty levels. On the actual exam, coding questions may take different forms and be more difficult.

# Example Questions

Q1: Given a matrix A and a vector b, solve the least-squares problem $Ax = b$ for $x = [x_1, x_2, x_3]$. Report $x_1$ (the first element of $x$), rounded to 2 decimal places.

```
import numpy as np

A = np.array([
    [ 0.0,  0.0, 1.0],
    [ 0.8,  0.0, 0.6],
    [-0.8,  0.0, 0.6],
    [ 0.0,  0.8, 0.6],
    [ 0.0, -0.8, 0.6],
])

b = np.array([150, 130, 50, 30, 150])
```

Q2: Given an RGB image represented as a floating-point array in numpy format with shape (height, width, 3), which statement below best describes what the code is doing? Note that the variable “input_image” is the given RGB image.

```
output = (input_image.max(axis=-1) + input_image.min(axis=-1)) / 2
```

- The code computes a saturation map, indicating for each pixel how far its color is from gray.
- The code converts the input image into a grayscale image, with a single intensity value per pixel.
- The code applies min-max normalization, rescaling the color values so that the output is a color image with the same shape as the input.
- The code computes one scalar, the midpoint between the darkest and brightest values found anywhere in the image.

Q3: Given the following code of the Grey-World algorithm to correct the color balance of an RGB image. Which of the following best fills in the blank, so that channel_mean holds the mean of each color channel (R, G, B) in awb_img with shape (H, W, 3)?

```
def apply_grey_world_algorithm(awb_img):
    """
    Apply the Grey-World algorithm to correct the color balance of an RGB image.
```

```
    This function assumes that under a white light source, the average color in a scene
should be grey.
    It corrects the image by scaling each channel so that the mean of each channel is equal.

    Args:
        awb_img: RGB image to be color corrected

    Returns:
        gw_img: Color corrected image using the Grey-World algorithm
    """

    # Fill in the line below
    channel_mean = ____________________

    target_mean = awb_img.mean()
    gw_img = awb_img / channel_mean * target_mean
    gw_img = gw_img.clip(0, 255).astype(np.uint8)

    return gw_img
```

- awb_img.mean(axis=(0, 1), keepdims=True)
- awb_img.mean(axis=(0, 1, 2), keepdims=True)
- awb_img.mean(axis=2, keepdims=True)
- awb_img.mean(axis=0, keepdims=True)

Q4: The following code runs without raising any error. But for every input image, the output is identical to the input (no color correction is ever applied). Which of the following best identifies the bug?

```
def apply_grey_world_algorithm(awb_img):
    """
    Apply the Grey-World algorithm to correct the color balance of an RGB image.

    This function assumes that under a white light source, the average color in a scene
should be grey.
    It corrects the image by scaling each channel so that the mean of each channel is equal.

    Args:
        awb_img: RGB image to be color corrected

    Returns:
        gw_img: Color corrected image using the Grey-World algorithm
    """

    channel_mean = awb_img.mean(axis=(0, 1, 2), keepdims=True)
    target_mean = awb_img.mean()
    gw_img = awb_img / channel_mean * target_mean
    gw_img = gw_img.clip(0, 255).astype(np.uint8)

    return gw_img
```

- target_mean should instead be an array with one value per channel. Currently, because it is a single scalar, the multiplication broadcasts incorrectly and silently leaves the image unchanged.
- gw_img divides the channel_mean before multiplying, but the algorithm requires multiplying before dividing. Doing this in the wrong order causes the values to round back to the original image.
- gw_img.clip(0, 255) re-clips every pixel back into its original range, which discards any correction that was applied earlier in the function.
- channel_mean is computed with axis=(0, 1, 2), which averages over the channel axis, collapsing it to a single scalar equal to the image's overall mean. Dividing and then multiplying by that same value cancels out.

Q5: Given a numpy array input L with shape (25, 3) and another numpy array input i with shape (25,). Assume matrix L has full column rank (rank 3). Which of the following best describes what the coding is doing?

```
import numpy as np
g, _, _, _ = np.linalg.lstsq(L, i, rcond=None)
```

- It computes the exact solution to $Lg = i$ by inverting matrix L to calculate the exact solution for $g$.
- It iteratively minimizes a loss function via gradient descent, so the returned g depends on a learning rate and number of iterations.
- It finds the g that solves the system $Lg = i$, and $g$ is the closest possible least-squares solution.
- It finds the eigenvector of L corresponding to its smallest eigenvalue, where g is the eigenvector of matrix $L^T L$.

Q6: Assume you are given an input PNG image and have access to its intrinsic albedo (albedo_img) and shading images (shading_img). You want to implement the function recolor_image(albedo_img, shading_img) to recolor an object to pure red (255, 0, 0) using four steps. Which ordering of these four steps best implements the recolor_image function?

A: Reconstruct the recolored image by multiplying the recolored albedo with the shading image.
B: Reconstruct the recolored image by dividing the recolored albedo from the shading image.
C: Loop over every pixel (x, y) in the albedo image. For each pixel, set it to RED if its albedo value is not BLACK; otherwise keep it BLACK.
D: Loop over every pixel (x, y) in the albedo image. For each pixel, set it to RED if its albedo value is not RED; otherwise keep it RED.
E: Define the RED and BLACK color constants in RGB format, and initialize an empty array with the same shape as the albedo image.
F: Convert the albedo image to grayscale.

- EFCB
- ECA
- EFDA
- EDB

# Answers

Q1
50.00

```
x, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
print("x_1:", round(x[0], 2))
```

Q2
The code converts the input image into a grayscale image, with a single intensity value per pixel.

Q3
awb_img.mean(axis=(0, 1), keepdims=True)

Q4
channel_mean is computed with axis=(0, 1, 2), which averages over the channel axis, collapsing it to a single scalar equal to the image's overall mean. Dividing and then multiplying by that same value cancels out.

Q5
It finds the g that solves the system $Lg = i$, and $g$ is the closest possible least-squares solution.

Q6
ECA